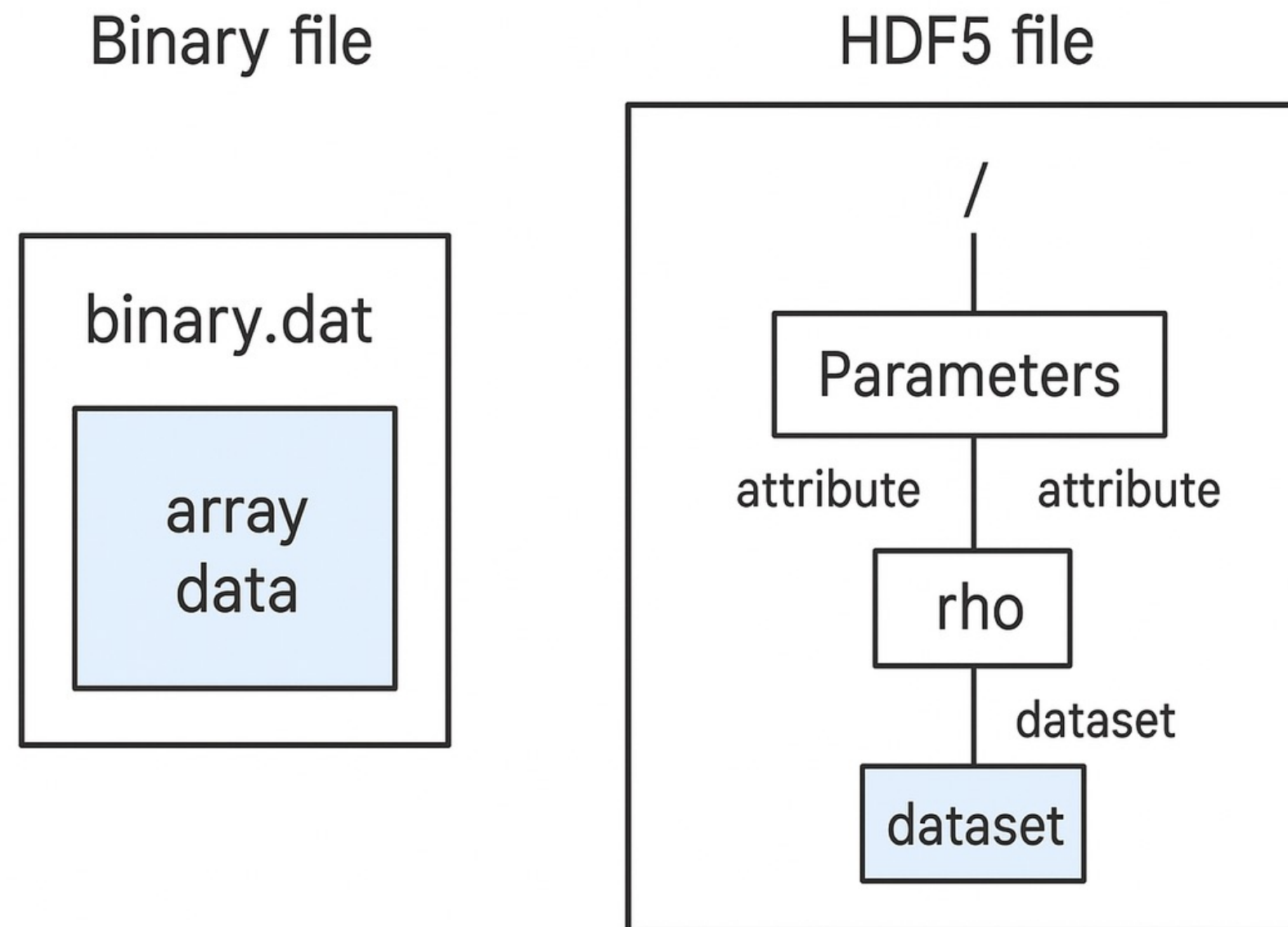


Introduction to HDF5 for Parallel Scientific Applications

Data Management and Parallel I/O with HDF5 in C

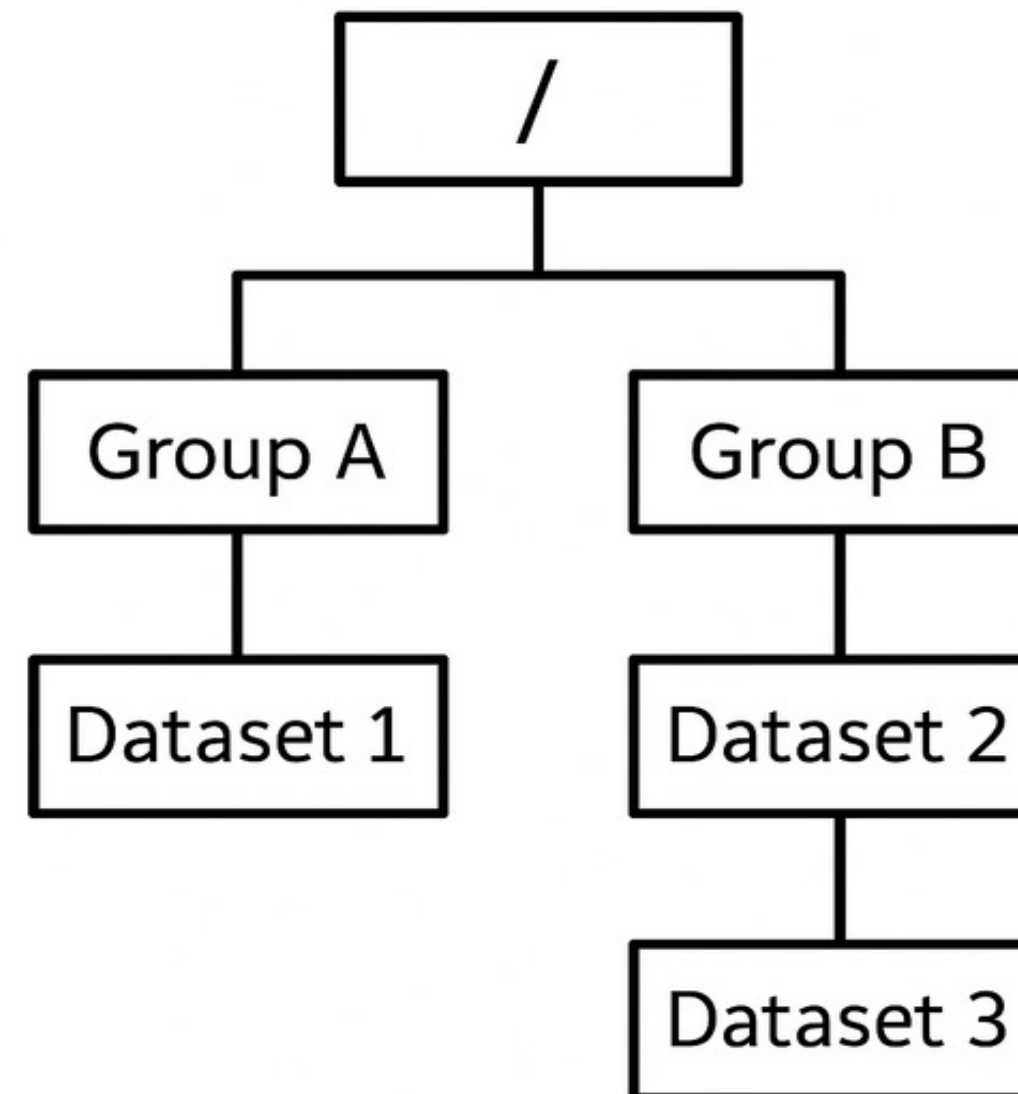
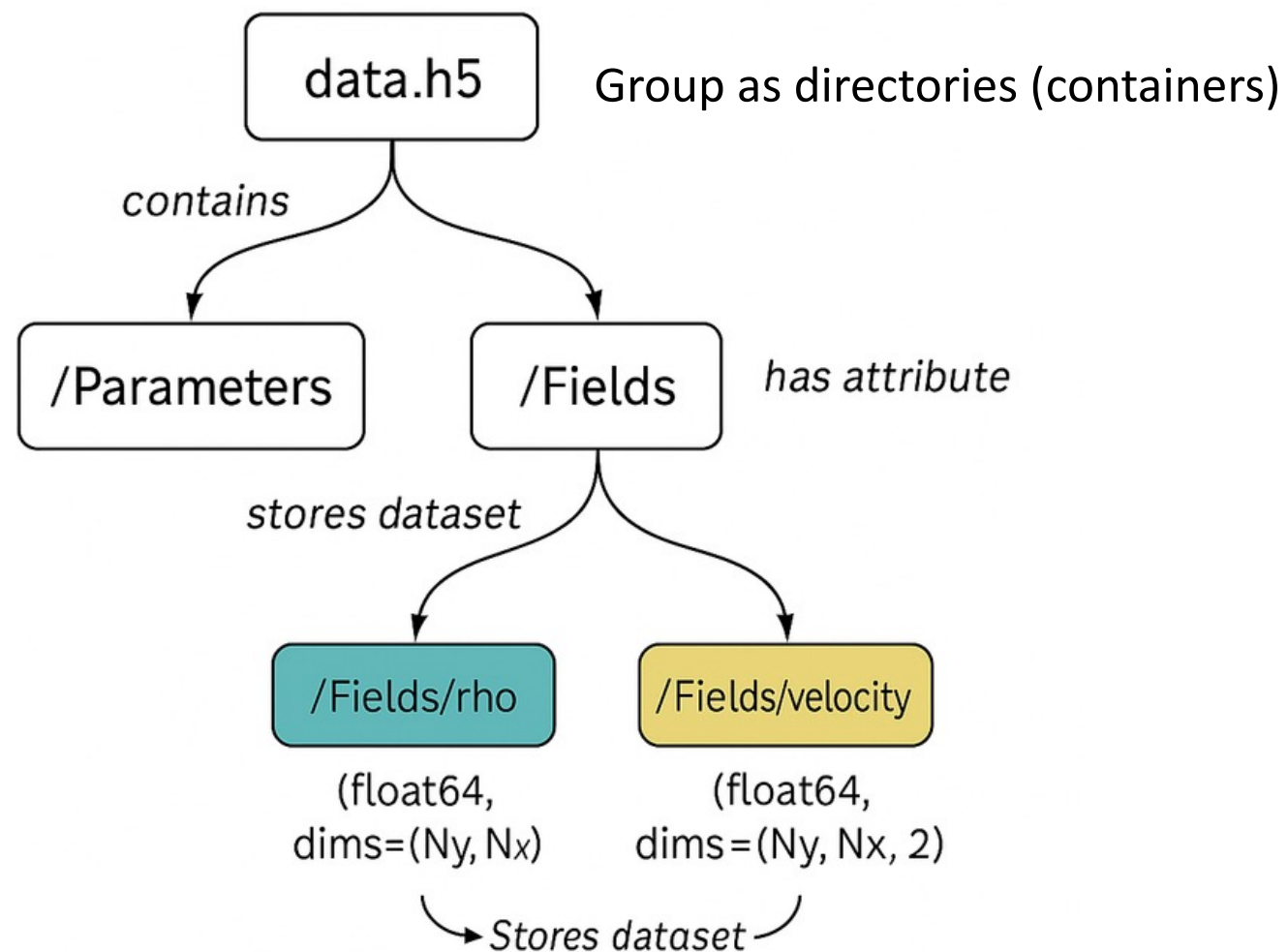
Motivation

- Challenges: binary dumps, lack of metadata, poor portability
- HDF5 pros: structured, portable, efficient, supports parallel I/O



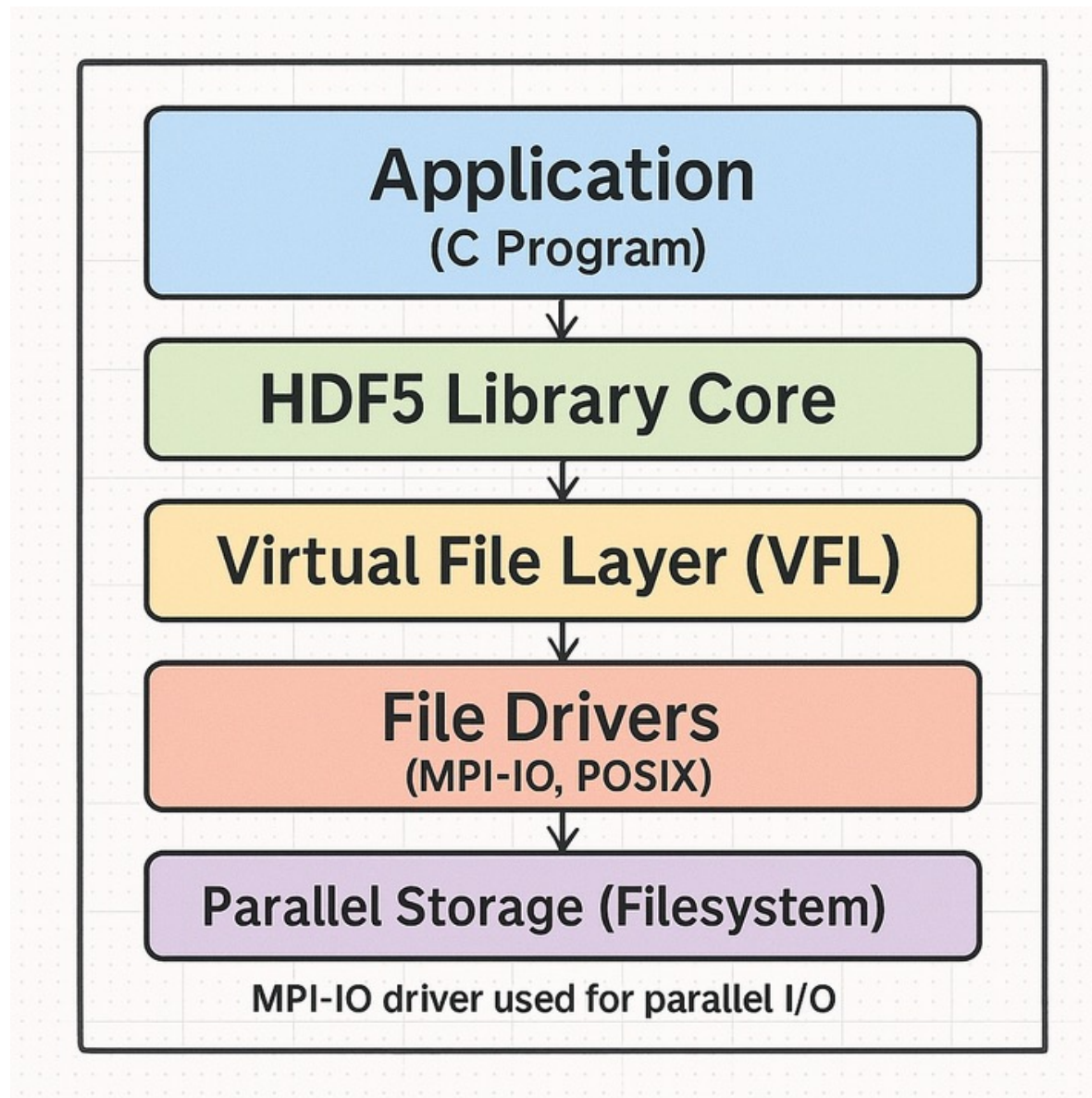
What is HDF5?

- Hierarchical Data Format for scientific data storage
- Stores datasets and groups with metadata attributes
- Portable and self-describing

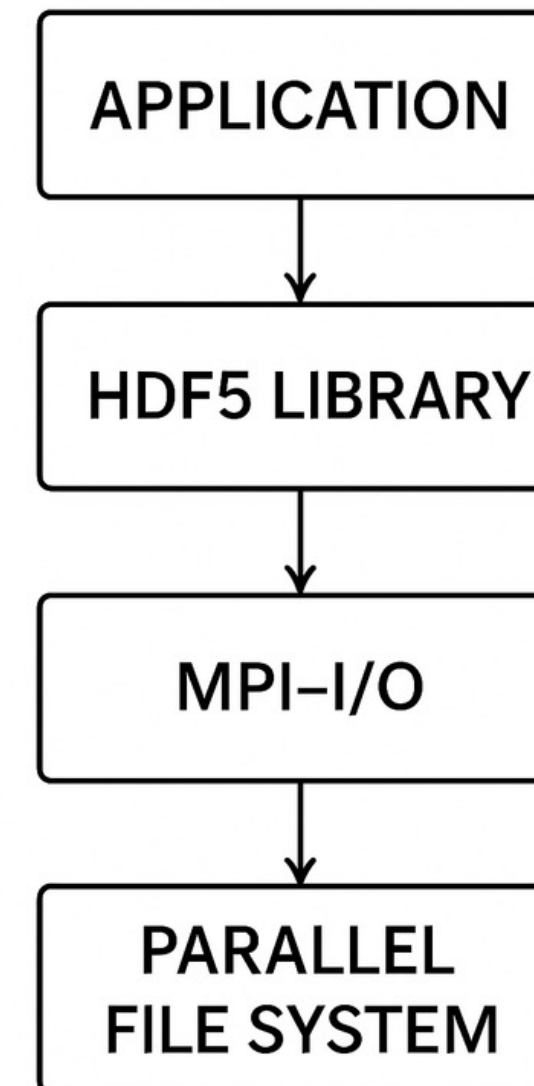


HDF5 Architecture Overview

- Application → HDF5 Library → Virtual File Layer → Storage (MPI-IO, POSIX)
- Parallel HDF5 uses MPI-IO as backend

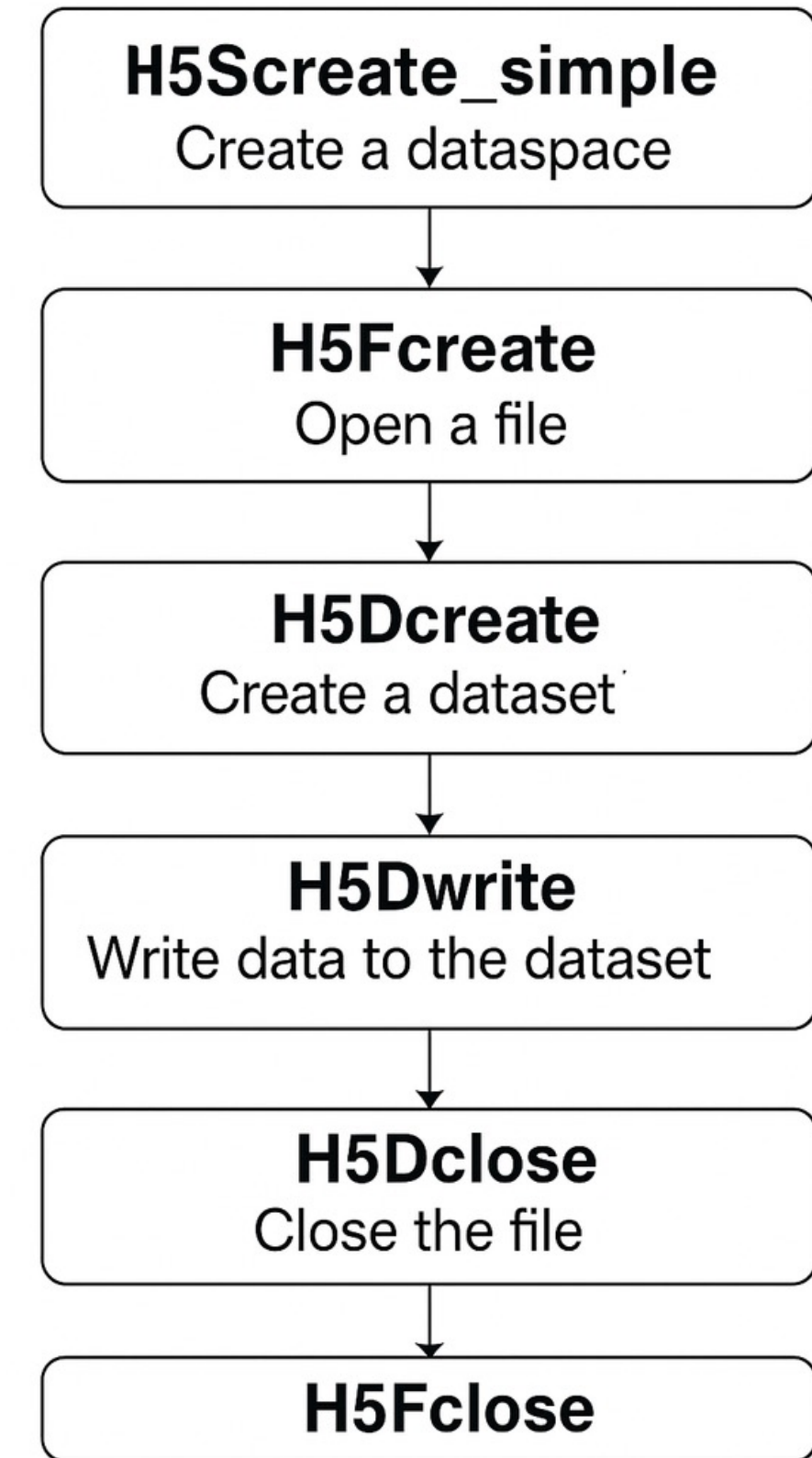


API CALL FLOW



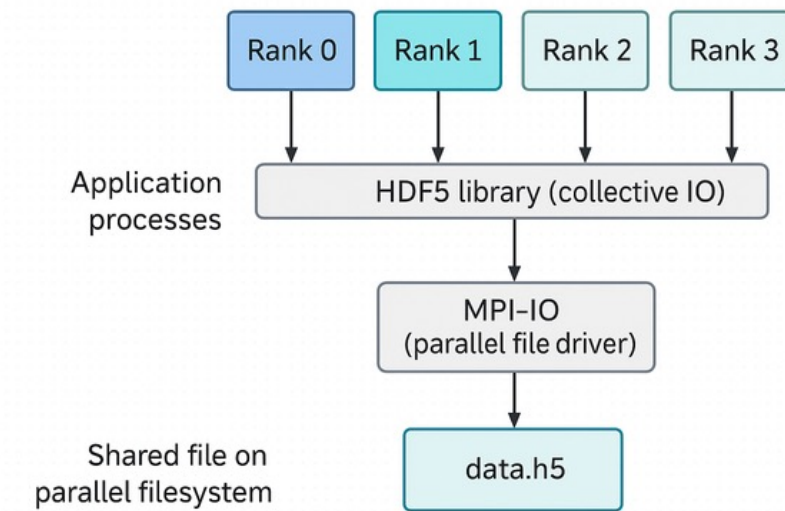
HDF5 Programming Model in C

- Example: create file, dataset, write data, close file
- Key functions: H5Fcreate, H5Dcreate, H5Screate_simple, H5Dwrite, H5Dclose, H5Fclose



Parallel HDF5 Overview

- Combine HDF5 + MPI for coordinated I/O
- Requires property list with H5Pset_fapl_mpio
- Collective vs. independent operations



Collective writes ensure consistency and performance

```
// Create an HDF5 file access property list
hid_t plist_id = H5Pcreate(H5P_FILE_ACCESS);

// Configure the property list for MPI-IO
// This links HDF5 I/O operations to the MPI communicator and info
H5Pset_fapl_mpio(plist_id, comm, info);

// Create or open the file collectively
hid_t file_id = H5Fcreate("data.h5", H5F_ACC_TRUNC, H5P_DEFAULT, plist_id);

// Clean up the property list (we don't need it after opening the file)
H5Pclose(plist_id);

// --- Additional parallel operations would go here ---
// For example: create dataspace, dataset, write data using H5Dwrite(...)

if (rank == 0) printf("Parallel HDF5 file created using MPI-IO\n");

// Close the file and finalize MPI
H5Fclose(file_id);
```



Writing a 1D Distributed Field (rho)

- Each process writes its subarray using hyperslabs

```
// ---- Create file access property list for MPI-IO ----
hid_t fapl = H5Pcreate(H5P_FILE_ACCESS);
H5Pset_fapl_mpio(fapl, MPI_COMM_WORLD, MPI_INFO_NULL);

// ---- Create file and global dataspace ----
hid_t file = H5Fcreate("data.h5", H5F_ACC_TRUNC, H5P_DEFAULT, fapl);
H5Pclose(fapl);

hsize_t gdim[2] = { N, N };          // global dataset size
hid_t filespace = H5Screate_simple(2, gdim, NULL);

// ---- Create dataset /rho ----
hid_t dset = H5Dcreate2(file, "/rho", H5T_NATIVE_DOUBLE, filespace, H5P_DEFAULT, H5P_DEFAULT, H5P_DEFAULT);

// ---- Define this rank's hyperslab in the FILE space ----
hsize_t offset[2] = { start, 0 };    // start row, start col
hsize_t count[2] = { local_rows, N }; // number of rows, number of cols

H5Sselect_hyperslab(filespace, H5S_SELECT_SET, offset, NULL, count, NULL);

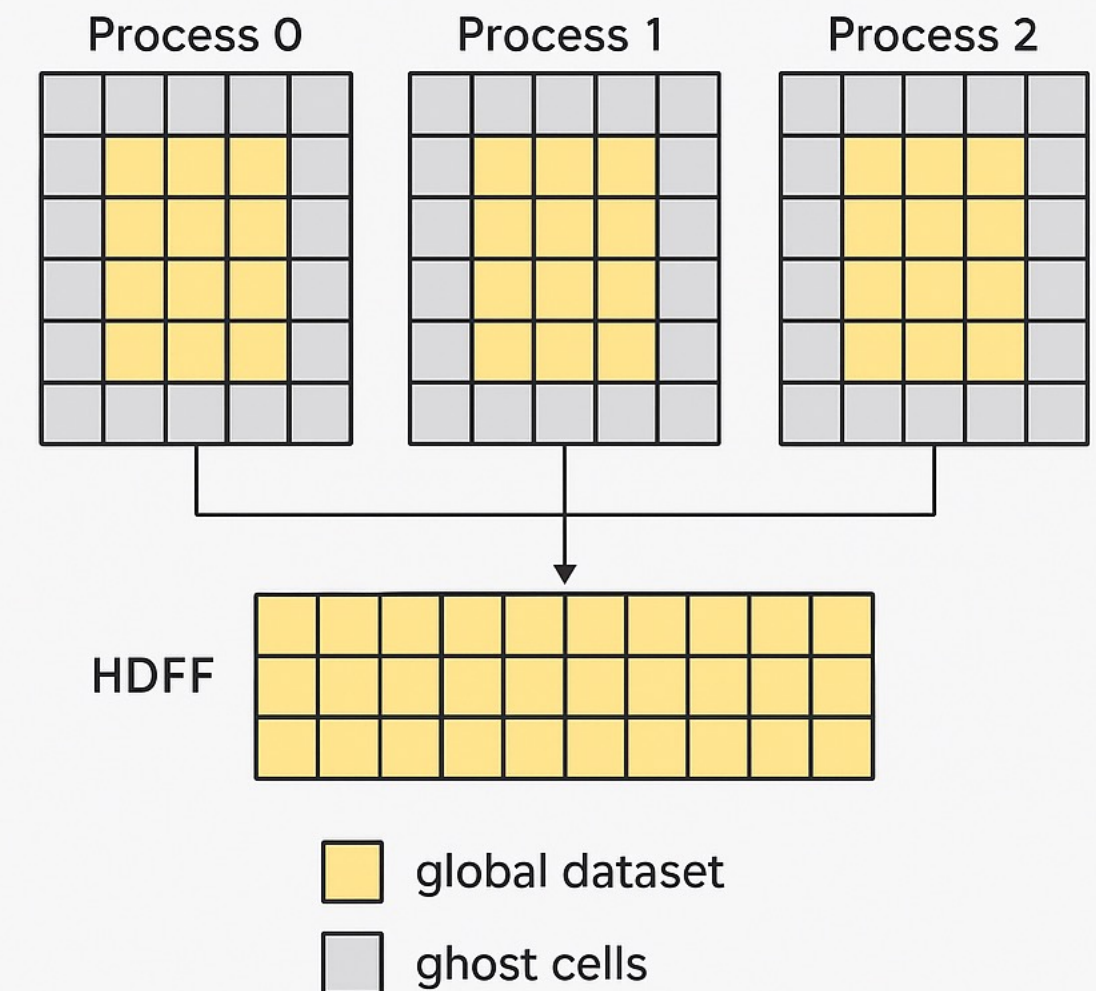
// ---- Define the MEMORY space for rho_local ----
hsize_t mdim[2] = { local_rows, NX };
hid_t memspace = H5Screate_simple(2, mdim, NULL);
hsize_t mem_off[2] = { 0, 0 };       // start at beginning of rho_local
H5Sselect_hyperslab(memspace, H5S_SELECT_SET, mem_off, NULL, count, NULL);

// ---- Collective write ----
hid_t xfer = H5Pcreate(H5P_DATASET_XFER);
H5Pset_dxpl_mpio(xfer, H5FD_MPIO_COLLECTIVE);

H5Dwrite(dset, H5T_NATIVE_DOUBLE, memspace, filespace, xfer, rho_local);

// ---- Cleanup ----
H5Pclose(xfer); H5Sclose(memspace); H5Sclose(filespace); H5Dclose(dset); H5Fclose(file);
```

Parallel Read & Reconstruct with Ghost Cells



Writing a 2D Distributed Field (rho)

- 2D Cartesian decomposition with `MPI_Cart_create`
- Select hyperslab region per process and write collectively



Concept of a Hyperslab

- A hyperslab is a contiguous or strided subset of a dataset.
- It defines a multidimensional slice using start, stride, count, and block.
- Used to describe what part of a dataset each process accesses.
- You define hyperslabs in both file and memory spaces.
- Use `H5Sselect_hyperslab()` to specify the region of interest.

- Example:
- `hsize_t start[2] = {y0, x0};`
- `hsize_t count[2] = {ny_local, nx_local};`
- `H5Sselect_hyperslab(filespace, H5S_SELECT_SET, start, NULL, count, NULL);`

Performance and Best Practices

- Use collective writes, chunking, compression
- Align datasets with file system blocks
- Prefer fewer large writes



References

- The HDF Group: <https://www.hdfgroup.org/solutions/hdf5/>
- HDF5 User's Guide & Parallel HDF5 Tutorial
- MPI Standard (MPI-IO section)
- LLNL HDF5 Parallel Tutorial
- A. Ching et al., Parallel I/O with HDF5 (LLNL Tech Report)



Summary & Takeaways

- HDF5 simplifies scientific data management
- Parallel HDF5 enables scalable I/O with MPI
- Key steps: define spaces → select hyperslabs → collective write
- Tools: HDFView, h5dump for inspection

